Assignment Report

Student: Denisenko Sofiia 狄苏菲
Student id: 1820249051
Course: Computer Organization and Architecture

Assignment №4

1.

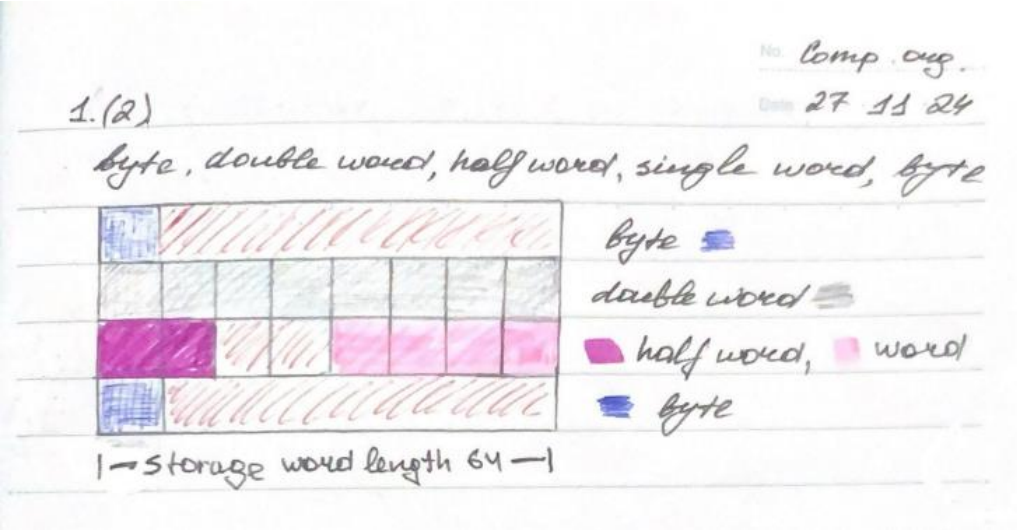
The word length of a certain machine is 64 bits, and the main memory is addressed by bytes. There are 4 types of data: byte, half word (16 bit), single word (32 bit), double word (64 bit). Please use a method that not only saves storage space, but also ensures that any data is read and written in a single access cycle, to store data of different lengths in the main memory (using big-endian)
(1) Show the restriction requirements of the address of different types of data in the main memory (that is, the requirement of the address for the first byte)

To ensure single-cycle access for all data types, addresses must be aligned to their respective sizes:

- **Byte:** No alignment restriction; can start at any address.
- **Half word (16-bit):** Address must be a multiple of 2 (even address). This ensures the 16-bit data is contained within a single memory location (two consecutive bytes) accessible in one operation.
- **Single word (32-bit):** Address must be a multiple of 4.
- **Double word (64-bit):** Address must be a multiple of 8.

So, the last three binary bits of a double word address must be 000 (such as 0, 8, 16, 24), the last two bits of a single word address must be 00(such as 0, 4, 8, 12, 16), and the last bit of a half word address must be 0 (such as 0, 2, 4, 6, 8).

(2) Draw a schematic diagram of storing the five data “byte, double word, half word, single word, and byte” in the main memory in sequence (the order cannot be changed)



2.

Assume the word length of a certain machine is 8 bits, and its address bus has16 bits. The 1K × 4 SRAM chips are used to form the main memory. The capacity of main memory is 4K × 8, then
(1) How many chips are needed?

SRAM chip capacity: 1K × 4 bits, meaning it can store 1024 words, each 4 bits wide.

Main memory capacity: 4K × 8 bits (4096 words, 8 bits per word).

Calculating the number of chips:

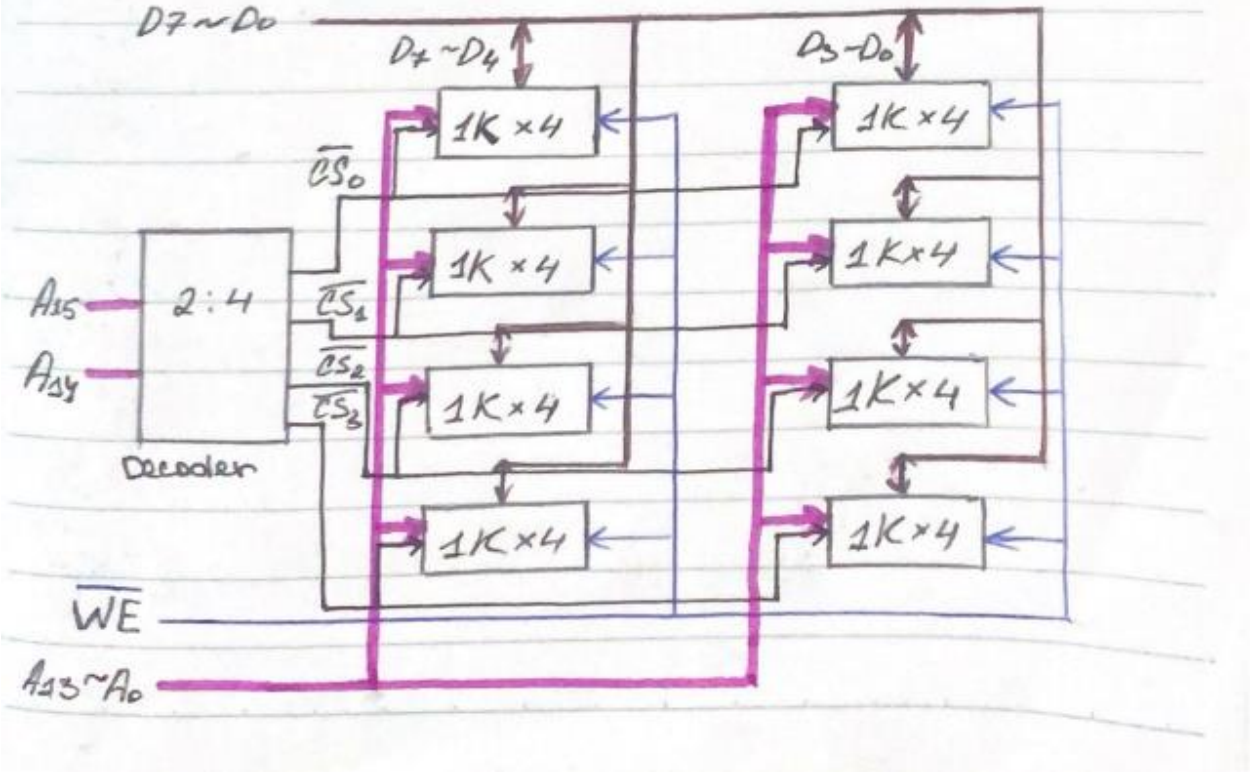
1) **Words per chip:** Each chip has 1024 words, each of 4 bits. To get 8-bit words, we will need two chips to make a complete 8-bit word from two 4-bit words.
2) **Chips needed for word size:** We need 2 chips to form an 8-bit word from two 4-bit words.
3) **Chips needed for total words:** To get 4096 words, we need 4096 words / 1024 words/chip = 4 chips. This assumes using 4 chips, each storing a portion of each of the 4096 words.
4) **Total chips:** Combining steps 2 and 3, the total number of chips required is **2 chips/word * 4 chips = 8 chips**.

Answer: 8 SRAM chips are needed.

(2) Draw the connection logic structure diagram of all chips when forming the main memory.

2 (2)

	Capacity	Address	Data
Memory	4K×8	16	8
Memory chip	1K×4	14	4



3.

Use some of the DRAM (16K*1) chips to form a 64KB memory (each chip memory matrix as 128*128). The read/write cycles of the memory is 0.5μs, the CPU must access the memory at least once within 1μs. The storage needs to be refreshed in every 2ms.

(1) Which refresh method is more reasonable?

The distributed refresh method. This method refreshes rows as they are accessed during normal read/write operations, ensuring that each row is refreshed at least once every 2ms without significantly impacting performance. This is preferable to a dedicated refresh cycle, which could interrupt CPU access. The distributed refresh method has no dead zone.

(2) What is the refresh interval between two adjacent rows?

Generally, the maximum refresh interval is 2ms, that is, all memory banks should be refreshed within 2 ms.
Calculation:
Total memory capacity: 64KB = 65536 bytes = 524288 bits (8 bits/byte).
DRAM chip capacity: 16K x 1 bit = 16384 bits.
Number of chips: 524288 bits / 16384 bits/chip = 32 chips.
Total refresh time: 2ms = 2000 μs.
Rows per chip: 128.
Total number of rows (across all chips): 128 rows/chip * 32 chips = 4096 rows.
If a distributed refresh strategy refreshes one row at a time across all chips:
Time per row refresh: 2000 μs / 4096 rows ≈ **0.49 μs**.

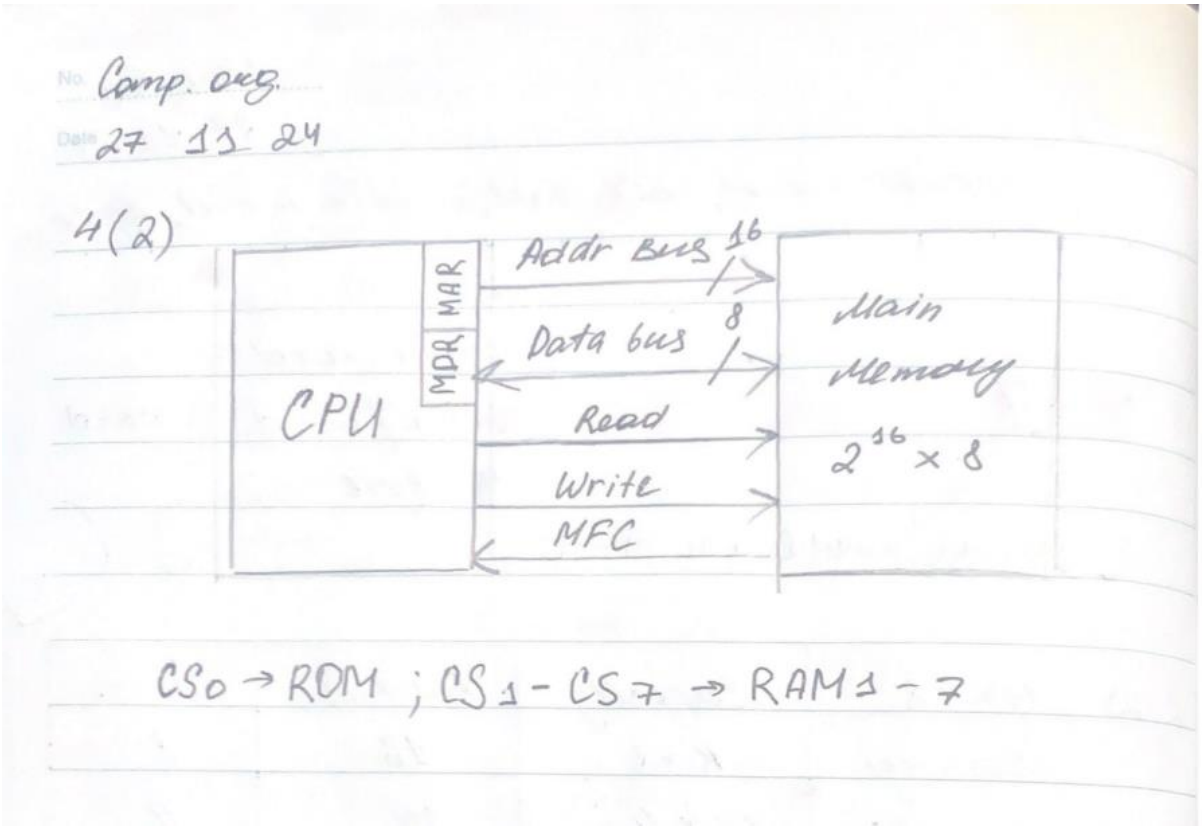
Answer: Refresh interval between adjacent rows approximately 0.49 μs.

4.

There is an 8-bit computer with 16-bit address. The main memory-related signals have MREQ (low-level allows memory access) and R/W (high level is read, low level is write). The address allocation is as follows: from 0 to 8191 is the system program area, which is composed of ROM chips; from 8192 to 32767 is the user program area; the last 2K address space is the system program work area, these two are composed of RAM chips. The memory is byte-addressed.
The following memory chips are available: 8K×8 ROM, 16K×1 SRAM, 2K×8 SRAM, 4K×8 SRAM, 8K×8 SRAM.
(1) Please select the chips to design the main memory of the machine.

System Program Area (ROM): This area requires 8192 bytes (8KB). We need one 8K x 8 ROM chip.
User Program Area (RAM): This area needs 32767 - 8192 + 1 = 24576 bytes (24KB). Using six 4K x 8 RAM chips.
System Program Work Area (RAM): This area needs 2048 bytes (2KB). A single 2K x 8 RAM chip is ideal for this section.

(2) Draw the connection logic structure diagram of the main memory, and pay attention to the chip selection logic and the connection with the CPU.



5.

Suppose the main memory capacity of a machine is 4MB, the Cache capacity is 16KB, each block contains 8 words, each word is 32 bits, and a four-way set associative mapping (that is, each set has four blocks). CPU firstly accesses the Cache, if it misses, then start to access the main memory. (1) What is the main memory address structure? (That is, which part is the zone number, which part is the group number, block number and address in block?)

First, we need to determine the number of bits required for each address field:

Main memory size: 4MB = 2^{22} bytes = 2^{25} words (since each word is 32 bits = 4 bytes). So, we need 25 bits to address the main memory.

Cache size: 16KB = 2^{14} bytes = 2^{17} words.

Block size: 8 words = 2^3 words. This needs 3 bits to address a specific word within a block.

Number of blocks per set: 4 (four-way set associative). This requires 2 bits ($\log_2 4$) to select a specific block within a set.

Number of sets: The total number of words in the cache (2^{17}) divided by the number of blocks per set (4) and the number of words per block (8) gives the

number of sets. So, $2^{17} / (4 * 8) = 2^{17-5} = 2^{12}$ sets. We need 12 bits to specify the set number.

The main memory address structure is:

Address (25 bits): Set Index (12 bits), Block Offset (2 bits), Word Offset (3 bits)

(2) The initial state of the Cache is empty, CPU reads 100 words from the 0th, 1, 2, ..., 99 units of the main memory in turn (the main memory reads one word at a time), and repeats 8 times, what is the hit rate?

The CPU reads 100 words (0-99) repeatedly 8 times:

First iteration: Accessing words 0-99 will bring in 13 blocks into the cache (because one block holds 8 words). There are 13 blocks/iteration that will be loaded into the cache.

Subsequent iterations: Since it's a four-way set-associative cache, it's possible for the same set to contain blocks from previous iterations. If there is a hit, we do not load the block from main memory again. The cache has space for 4 blocks in each set. Thus, in subsequent iterations, all access will result in cache hits. Because the address structure ensures that the same sets are accessed each iteration, there are some potential cache hits.

The number of misses and hits.

- **Misses:** 13 blocks are loaded from main memory in the first iteration. Subsequent iterations will have 0 misses if the same sets are reused as the same sets are accessed.
- **Hits:** All access in the second and subsequent iterations will be hits.
- **Total accesses:** 100 words/iteration * 8 iterations = 800 words.

The number of misses in the first iteration is equal to 13 blocks. The remaining iterations will all be hits. So, the number of hits is 800-13=787.

The hit rate is 787/800 = 98.375%.

(3) If the speed of the Cache is 6 times of the main memory, how much is the speed increased compared with the non-Cache memory?

Let's assume the time to access the main memory is Tm. The time to access the cache is Tc = Tm / 6.

Without cache: The time to access 800 words would be 800 * Tm.

With cache: There are 13 misses, each taking Tm time, and 787 hits, each taking Tc time. The total time with the cache is (13 * Tm) + (787 * Tm/6) =144.17 * Tm.

Speedup: (800 * Tm) / (144.17 * Tm) ≈ 5.55

The speed increase with the cache is approximately 5.55 times compared to the non-cache memory.